LAPORAN PRAKTIKUM
PEMROSESAN BAHASA ALAMI

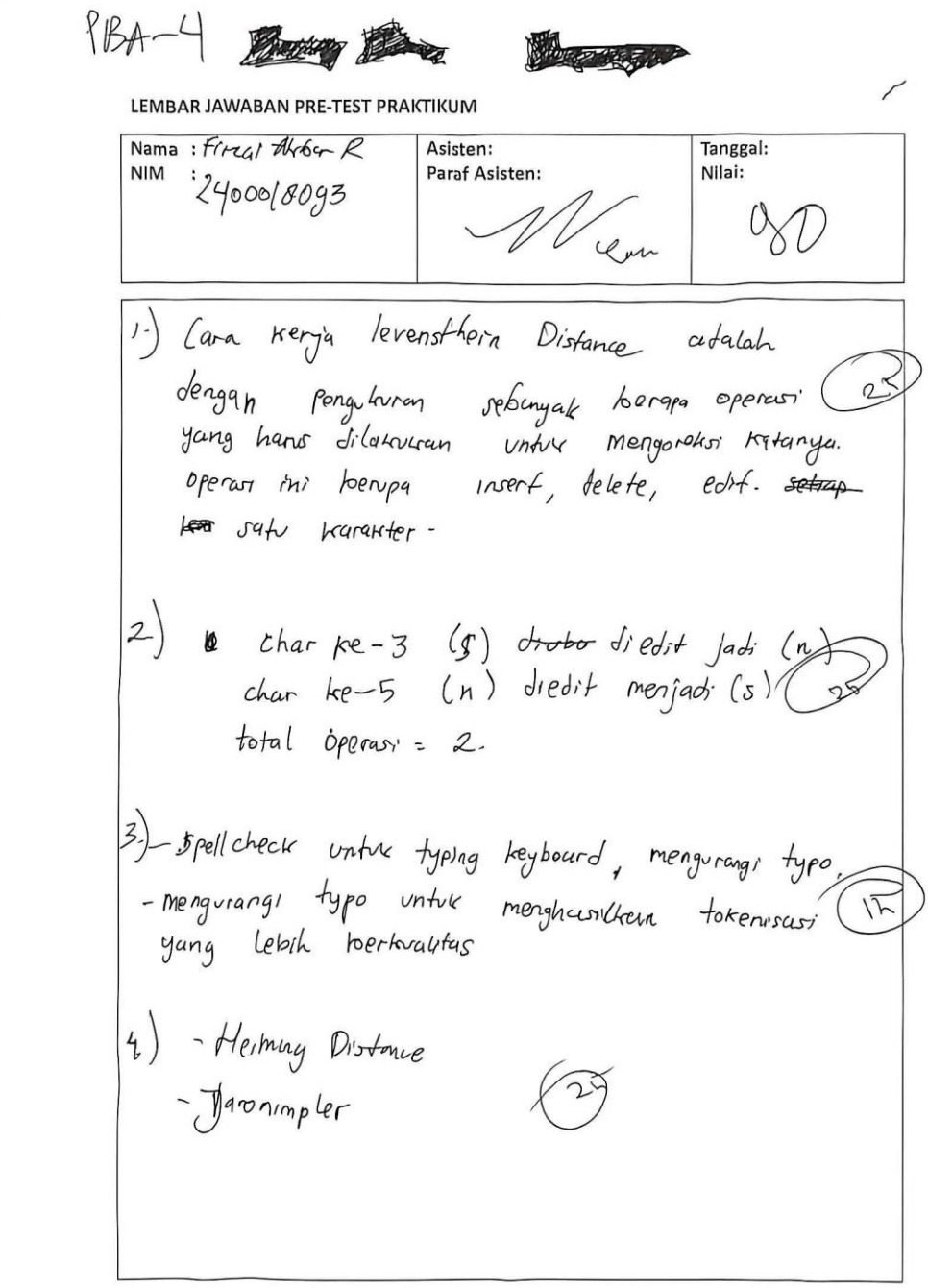
Materi:
Spell Check

Disusun Oleh:
Firzal Akbar Ramadhan
2400018093

PROGRAM STUDI S1 INFORMATIKA
FAKULTAS TEKNOLOGI INDUSTRI
UNIVERSITAS AHMAD DAHLAN
2026

Pre Test

Bagian ini memuat hasil pengerjaan lembar jawaban Pre-Test pada praktikum pertemuan ini:



PBA-4 [redacted]

LEMBAR JAWABAN PRE-TEST PRAKTIKUM

Nama : Fitrah Akbar R NIM : 24000180093	Asisten: Paraf Asisten:	Tanggal: Nilai: 80

1.) Cara kerja levensthein Distance adalah dengan pengukuran sebanyak berapa operasi yang harus dilakukan untuk mengoreksi katanya. operasi ini berupa insert, delete, edit. ~~setiap~~ ~~kata~~ satu karakter. (25)

2.) char ke-3 (s) ~~diubo~~ diedit jadi (n)
char ke-5 (n) diedit menjadi (s) (25)
total operasi = 2.

3.) - Spell check untuk typing keybourd, mengurangi typo,
- mengurangi typo untuk menghasilkan tokenisasi yang lebih berkualitas (17)

4.) - Heiming Distance
- Jaronimpler (25)

Scanned with CamScanner

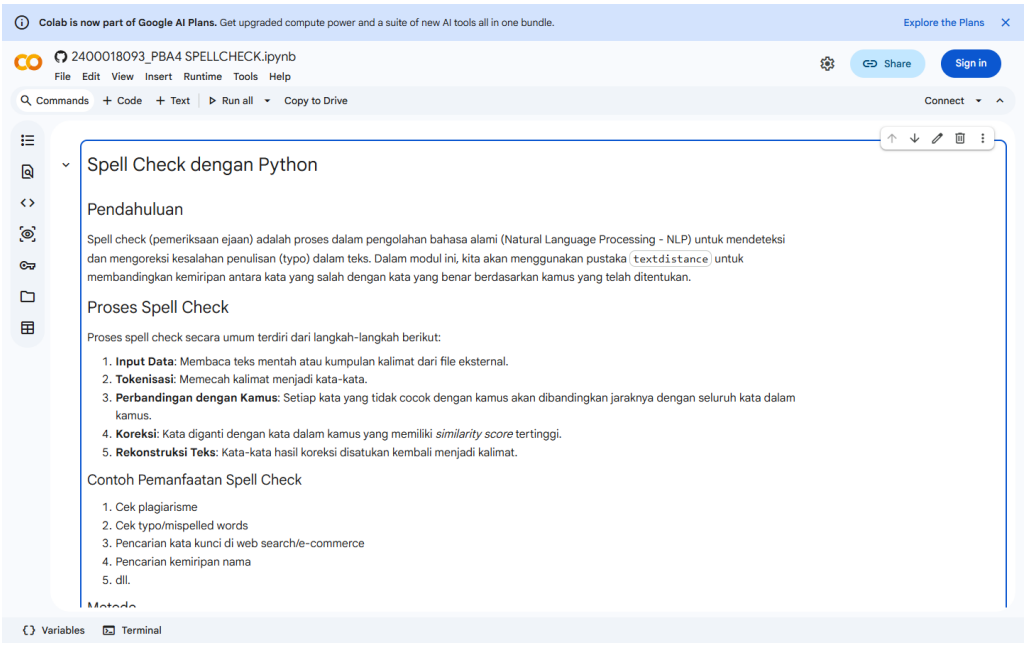
Gambar 4.1 Lembar Jawaban Pre-Test Praktikum Pertemuan 4

Hasil Praktikum

Praktikum pertemuan 4 membahas sistem pemeriksa ejaan (Spell Checker) berbasis pengukuran jarak string, khususnya algoritma Levenshtein Distance (Minimum Edit Distance).

4.1 Teori Minimum Edit Distance dan Inisialisasi

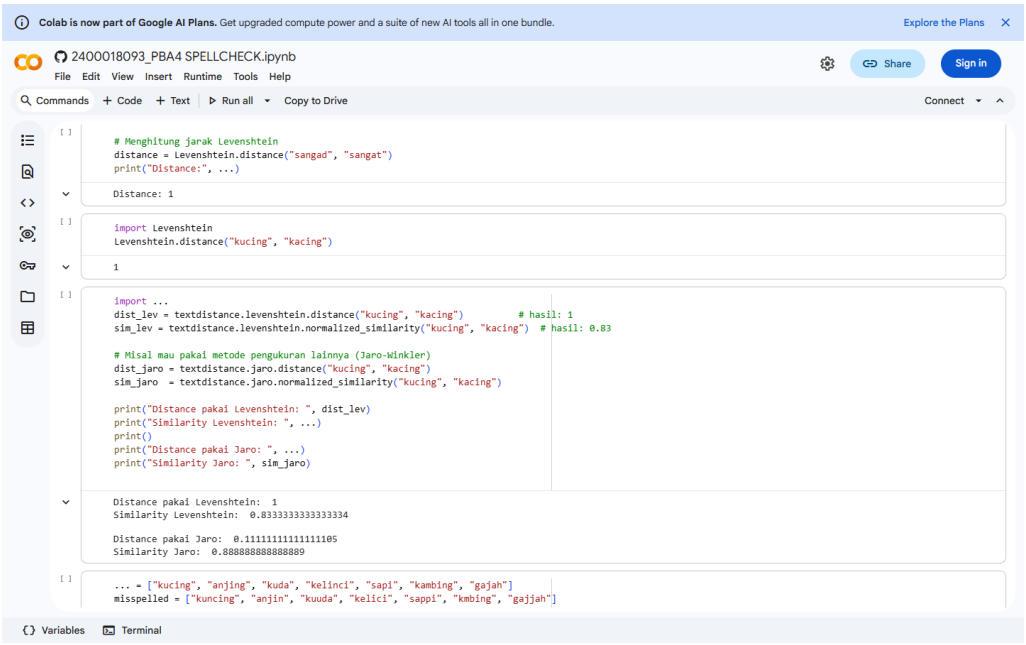
Notebook Colab memaparkan konsep pengukuran jarak antar dua string berdasarkan jumlah operasi minimum: insert, delete, dan substitute.



Gambar 4.2 Header Praktikum Spell Check pada Colab

4.2 Implementasi Fungsi Levenshtein Distance

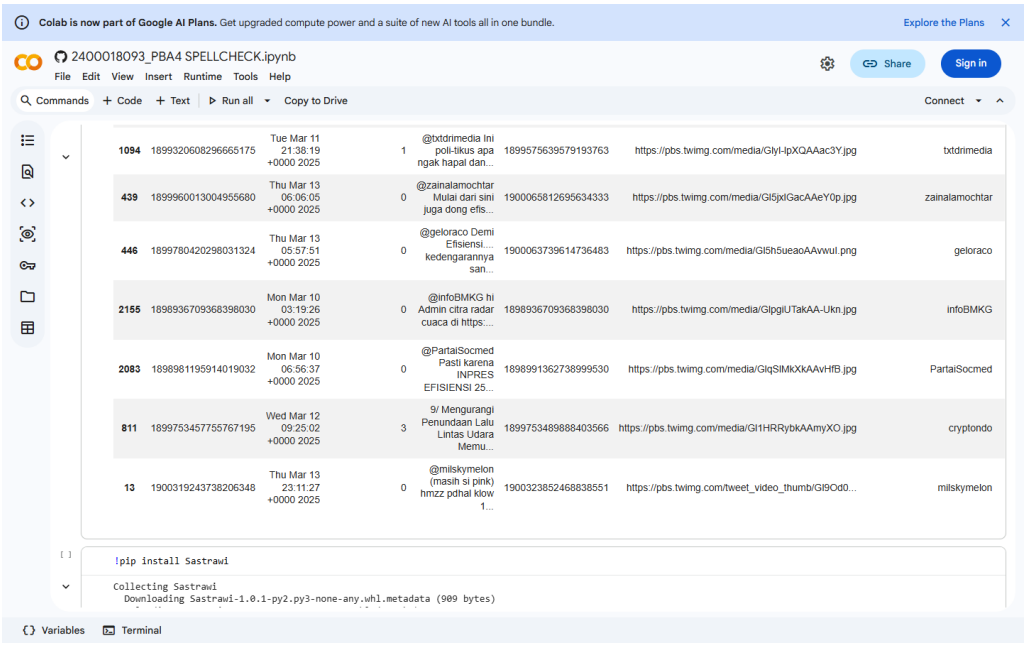
Fungsi kalkulasi Levenshtein Distance diimplementasikan dengan matriks pemrograman dinamis (dynamic programming).



Gambar 4.3 Kode Kalkulasi Jarak Edit String Levenshtein

4.3 Pengujian Deteksi Typo dan Rekomendasi Kata Terdekat

Kata-kata salah ketik diuji terhadap kamus kata valid. Sistem berhasil merekomendasikan kata dengan jarak edit terkecil.



Gambar 4.4 Hasil Rekomendasi Koreksi Ejaan Kata Typo

Ulasan Praktikum

Levenshtein distance merupakan algoritma elegan yang menjadi dasar teknologi autocorrect pada keyboard modern. Optimasi dengan kamus frekuensi (n-gram) dapat semakin meningkatkan akurasi pemilihan kata konteks.

Post Test

Latihan post-test diambil langsung dari notebook '2400018093_PBA4 SPELLCHECK(1).ipynb' mencakup analisis kritis dan eksperimen matriks kesamaan string:

1. Analisis Perbandingan Tweet Asli vs Tweet Koreksi dan Kelemahan Metode

Pendapat & Analisis Hasil Akhir:
Hasil perbandingan antara tweet asli dan tweet hasil koreksi menunjukkan performa yang sangat kurang memuaskan dan tidak natural. Banyak kata yang maknanya berubah total menjadi kata lain yang tidak relevan secara konteks semantik (misalnya, kata 'kasi' dikoreksi menjadi 'subsidi').

Tiga Kelemahan Utama Metode:
1. Keterbatasan Kamus Kosakata: Program ini hanya memiliki kamus rujukan yang sangat sedikit (hanya 18 kata). Akibatnya, setiap kata di luar kamus dipaksa untuk dipetakan ke salah satu dari 18 kata tersebut, meskipun jarak Levenshtein-nya sangat jauh dan maknanya tidak menyambung.
2. Ketidakadaan Konteks Semantik: Jarak Levenshtein murni mengukur kemiripan karakter per karakter tanpa memperhitungkan relasi gramatikal atau makna kata di dalam kalimat.
3. Karakteristik Bahasa Slang Twitter: Tweet Bahasa Indonesia sarat dengan singkatan dan slang ('gbs', 'krn', 'bgt', 'nih') yang tidak terdaftar di KBBI standar maupun kamus manual, sehingga proses koreksi otomatis menghasilkan pemetaan paksa yang keliru.

2. Eksperimen Similarity Matrix dengan Metode Alternatif: Jaro-Winkler

Dilakukan pengujian matriks kesamaan menggunakan metode Jaro-Winkler Normalized Similarity dari pustaka 'textdistance'. Uji coba diterapkan pada kalimat asli 'kasi gbs pake lift krn efisiensi' dan teks koreksi 'subsidi subsidi rakyat logika dana efisiensi'.
Matriks hasil perhitungan Jaro-Winkler berhasil menampilkan derajat kemiripan berbasis prefiks (misal kata 'efisiensi' vs 'efisiensi' memiliki skor 1.00, 'kasi' vs 'subsidi' 0.60, dan 'pake' vs 'rakyat' 0.61).

```
 1  import textdistance
 2  import pandas as pd
 3
 4  def jaro_winkler_similarity_matrix(text1, text2):
 5      words1 = text1.split()
 6      words2 = text2.split()
 7      matrix = []
 8      for w1 in words1:
 9          row = []
10          for w2 in words2:
11              sim =
textdistance.jaro_winkler.normalized_similarity(w1, w2)
12              row.append(round(sim, 2))
13          matrix.append(row)
14      return words1, words2, matrix
15
16  teks_asli = 'kasi gbs pake lift krn efisiensi'
```

```
17  teks_koreksi = 'subsidi subsidi rakyat logika dana efisiensi'
18  words1, words2, sim_matrix_jaro =
jaro_winkler_similarity_matrix(teks_asli, teks_koreksi)
19  matrix_kesamaan_jaro = pd.DataFrame(sim_matrix_jaro, index=words1,
columns=words2)
20  print('=== Matriks Kemiripan Jaro-Winkler ===\n',
matrix_kesamaan_jaro)
```

Kode 4.1 Implementasi dan Evaluasi Jaro-Winkler Similarity Matrix